

```
In [1]: import pandas as pd
import numpy as np
import sklearn
```

```
In [2]: def draw_roc_from_data(df, test_size_val=0.2, plt_name=None, file_name=None):
    # get feature and target
    df_feature = df.copy(deep=True)
    df_feature.drop(columns=['target'], inplace=True)
    df_target = df['target'].copy(deep=True)
    df_feature.head(3)
    # split dataset
    x_train, x_test, y_train, y_test = sklearn.model_selection.train_test_split(
    # train model and predict
    log_reg = sklearn.linear_model.LogisticRegression(random_state=0, solver='ne
    log_reg.fit(x_train, y_train)
    train_predict_probs = log_reg.predict_proba(x_test)
    probs = train_predict_probs
    y_test = y_test
    draw_auc_with_prob( probs, y_test, plt_name, file_name )

#####
def draw_auc_with_prob( probs, y_test, plt_name=None, file_name=None ):
    preds = probs[:,1]
    fpr, tpr, threshold = sklearn.metrics.roc_curve(y_test, preds)
    roc_auc = sklearn.metrics.auc(fpr, tpr)

    # method 1: plt
    import matplotlib.pyplot as plt
    if plt_name is not None:
        title_name = plt_name + ': ROC Curve'
    else:
        title_name = 'ROC Curve'

    plt.figure(figsize=(10,6))
    plt.rcParams.update({'font.size': 16})
    plt.grid(color='grey', alpha=0.5, linestyle='--', linewidth=0.5);
    plt.title(title_name)
    plt.plot(fpr, tpr, color='tab:blue', label = 'AUC = %0.2f' % roc_auc)
    plt.legend(loc = 'lower right')
    plt.plot([0, 1], [0, 1], color='tab:orange', linestyle='--')
    plt.xlim([-0.02, 1])
    plt.ylim([0, 1.02])
    plt.ylabel('True Positive Rate')
    plt.xlabel('False Positive Rate')
    if file_name is not None:
        plt.savefig(file_name+'.png');
        plt.savefig(file_name+'.eps');
    plt.show()
```

```
In [ ]:
```

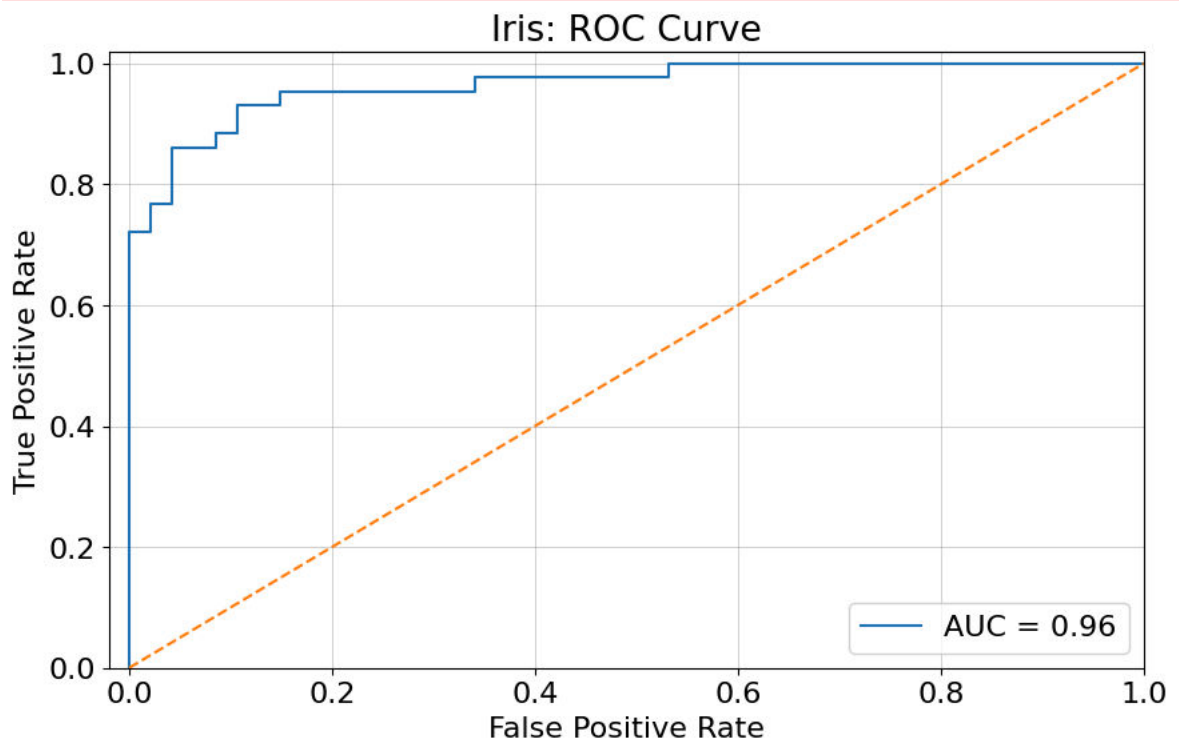
```
In [3]: # dataset: iris
from sklearn.datasets import load_iris
iris = load_iris()
df_raw = pd.DataFrame(np.c_[iris['data'], iris['target']],
                      columns= np.append(iris['feature_names'], ['target']))
```

```

df_raw.head(3)
#####
df = df_raw.copy(deep=True)
df = df[(df['target']==1)|(df['target']==2)];
df["target"] = df["target"] - 1
df_feature = df.copy(deep=True)
df_feature.drop(columns=['target'],inplace=True)
df_target = df['target'].copy(deep=True)
df_feature.head(3)
test_size_val=0.9
plt_name = 'Iris'
file_name = 'l6_roc_iris_0'+str(int(test_size_val*10))
draw_roc_from_data(df, test_size_val,plt_name,file_name)

```

The PostScript backend does not support transparency; partially transparent artists will be rendered opaque.

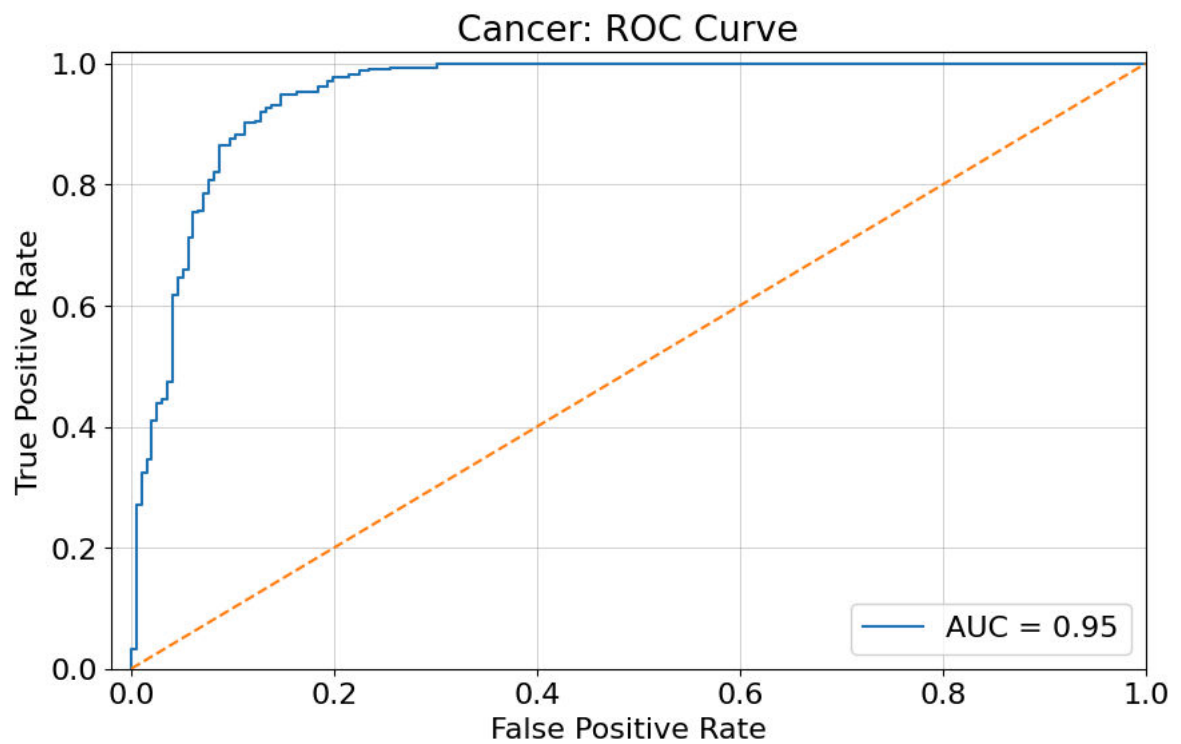


```

In [4]: # dataset: cancer
from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer()
df = pd.DataFrame(np.c_[cancer['data'], cancer['target']],
                  columns= np.append(cancer['feature_names'], ['target']))
df.head(3)
#####
df_feature = df.copy(deep=True)
df_feature.drop(columns=['target'],inplace=True)
df_target = df['target'].copy(deep=True)
df_feature.head(3)
test_size_val=0.9
plt_name = 'Cancer'
file_name = 'l6_roc_cancer_0'+str(int(test_size_val*10))
draw_roc_from_data(df, test_size_val,plt_name,file_name)

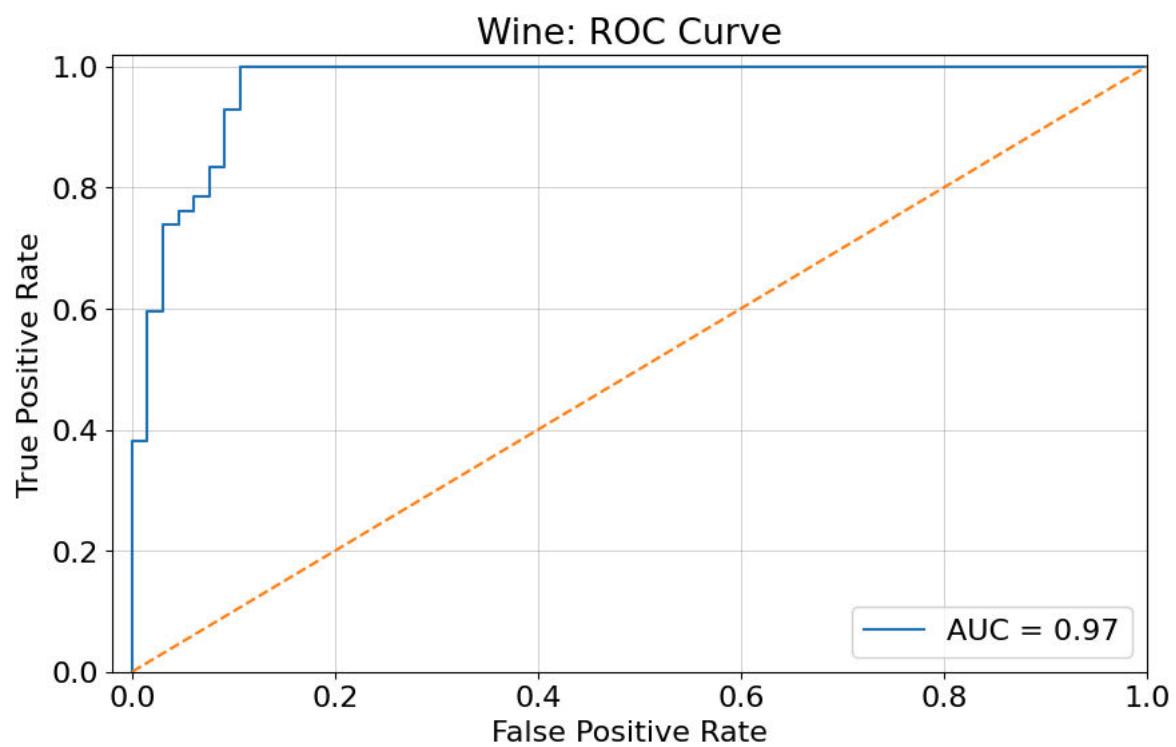
```

The PostScript backend does not support transparency; partially transparent artists will be rendered opaque.



```
In [5]: # dataset: wine
from sklearn.datasets import load_wine
wine = load_wine()
df_raw = pd.DataFrame(np.c_[wine['data'], wine['target']],
                      columns=np.append(wine['feature_names'], ['target']))
df_raw.head(3)
#####
df = df_raw.copy(deep=True)
df = df[(df['target']==1)|(df['target']==2)];
df["target"] = df["target"] - 1
test_size_val=0.9
plt_name = 'Wine'
file_name = 'l6_roc_wine_0'+str(int(test_size_val*10))
draw_roc_from_data(df, test_size_val,plt_name,file_name)
```

The PostScript backend does not support transparency; partially transparent artists will be rendered opaque.



In []: